Summa

Tensor contractions written as on paper: the package finds the order and the kernels

Everything the package does, step by step: every output is computed by the code printed above it

Nazzareno Africani · SISSA · version 1.0.0 · November 2026

Summa is a Wolfram Language package for tensor contractions, written with the syntax of `numpy.einsum`. A contraction is written as on paper, by the indices of every factor: `"ij,jk->ik"` is $\sum_j a_{ij} b_{jk}$. How to compute it is left to the package: it chooses the order of the pairwise contractions, which in a network of tensors can change the cost by orders of magnitude, and the kernel of each, BLAS or a vectorised one, with no transposes, reshapes or intermediate arrays written by hand. An index can appear any number of times, in one tensor or in many. This document goes through all the package does, from a matrix product to tensor networks and a Kubo formula: the syntax; indices repeated at will; the order of the contractions and the pictures of a network; compiled einsums for loops; symbolic, exact, complex and sparse arrays; computations of physics; how a contraction runs and how the package is checked. The code of each section uses only what the previous ones introduced.

Contents

1 Written as on paper **2**
- 1.1 One string for the whole contraction . . . 2
- 1.2 The implicit output . . . 3
- 1.3 Diagonals . . . 3
- 1.4 The ellipsis and broadcasting . . . 3
- 1.5 Scalars . . . 4
- 1.6 Labels of any kind . . . 4

2 Indices repeated at will **5**
- 2.1 One index in three tensors . . . 5
- 2.2 Batch indices . . . 5
- 2.3 The CP reconstruction . . . 6
- 2.4 Shared and kept . . . 6

3 The order of the contractions **6**
- 3.1 What a path costs . . . 6
- 3.2 Strategies . . . 7
- 3.3 The norm of a matrix product state . . . 7

4 Pictures of a network **8**
- 4.1 Bonds, open legs, traces, sums and hyperedges . . . 8
- 4.2 Dimensions and the path . . . 10
- 4.3 The contraction tree . . . 11

5 Compiled einsums and the plan cache **11**

6 Symbolic, exact, complex and sparse arrays 12

7 Physics 13
7.1 The partition function of the Ising model . . . 13
7.2 Matrix product states, exactly . . . 15
7.3 A Kubo formula . . . 15
7.4 A loop of propagators . . . 18

8 How a contraction runs 18

9 How it is checked 20

1 Written as on paper

1.1 One string for the whole contraction

The package is loaded with

```
Needs["Summa`"]
```

Each operand gets a subscript, a string of one-character indices; the subscripts are separated by commas, and the arrow is followed by the indices of the result. An index that appears in the operands and not in the result is summed over: `Einsum["ij,jk->ik", m, n]` is $\sum_j m_{ij} n_{jk}$, the matrix product,

```
m = {{1, 2}, {3, 4}}; n = {{0, 1}, {1, 1}};
Einsum["ij,jk->ik", m, n]
```

```
{{2, 3}, {4, 7}}
```

```
Einsum["ij,jk->ik", m, n] == m . n
```

```
True
```

How it is computed, here by one call of `Dot`, is the business of the package, for this contraction and for every one below. One operand is enough for a transpose, a trace, a diagonal and sums over rows or over all entries:

```
{Einsum["ij->ji", m], Einsum["ii->", m], Einsum["ii->i", m], Einsum["ij->i", m],
 Einsum["ij->", m]}
```

```
{{{1, 3}, {2, 4}}, 5, {1, 4}, {3, 7}, 10}
```

Two vectors give the inner and the outer product:

```
{Einsum["i,i->", {1, 2}, {3, 4}], Einsum["i,j->ij", {1, 2}, {3, 4}]}
```

```
{11, {{3, 4}, {6, 8}}}
```

1.2 The implicit output

Without the arrow, the result has the indices that appear exactly once, in alphabetical order, as in numpy: `"ij,jk"` is the matrix product, `"ba"` the transpose and `"ii"` the trace:

```
{Einsum["ij,jk", m, n] == m . n, Einsum["ba", m] == Transpose[m], Einsum["ii", m]}
```

```
{True, True, 5}
```

1.3 Diagonals

An index repeated within one operand takes a diagonal: `"iij->ij"` is t_{iij}, and `"iii->i"` the diagonal of a cube. The entries of `digits` are $100i + 10j + k$, so the results show which ones were taken:

```
digits = Array[100 #1 + 10 #2 + #3 &, {3, 3, 3}];
{Einsum["iij->ij", digits], Einsum["iii->i", digits]}
```

```
{{{111, 112, 113}, {221, 222, 223}, {331, 332, 333}}, {111, 222, 333}}
```

1.4 The ellipsis and broadcasting

`...` stands for the leading indices that are not named, as in numpy; here a stack of five 2×3 matrices times one 3×4 matrix:

```
SeedRandom[1];
stackA = RandomInteger[9, {5, 2, 3}]; stackB = RandomInteger[9, {3, 4}];
Dimensions[Einsum["...ij,...jk->...ik", stackA, stackB]]
```

$$\{5, 2, 4\}$$

Axes of size 1 are broadcast: the single row of the second operand is paired with every row of the first:

```
Einsum["...i,...i->...", {{1, 2}, {3, 4}, {5, 6}}, {{1, 1}}]
```

$$\{3, 7, 11\}$$

1.5 Scalars

An empty subscript is a scalar operand. Any argument that is not a list or a `SparseArray`, `Sqrt[2]` or `a + b` as well, is a scalar:

```
{Einsum[",ij->ij", Sqrt[2], m], Einsum[",i,i->", a + b, {1, 2}, {3, 4}]}
```

```
{{{Sqrt[2], 2*Sqrt[2]}, {3*Sqrt[2], 4*Sqrt[2]}}, 11*(a + b)}
```

1.6 Labels of any kind

In the list syntax the subscripts are lists, and an index can be any expression: a string, an integer, a symbol with arguments. The string syntax also takes Unicode letters, spaces and the arrow →:

```
Einsum[{{"bond1", "x"}, {"x", "bond2"}} -> {"bond2", "bond1"}, m, n]
```

```
{{2, 4}, {3, 7}}
```

```
Einsum["αβ, βγ → γα", m, n]
```

```
{{2, 4}, {3, 7}}
```

Indices with arguments, α_k and σ_k written `α[k]` and `σ[k]`, let a program write the subscripts of a network of any size (section 3.3).

2 Indices repeated at will

On paper an index appears in as many factors as needed: $\sum_i x_i y_i z_i$, or a batch index b that labels independent products and stays in the result, as in $\sum_j L_{bij} R_{bjk}$. Summa allows an index any number of times, within one tensor and across tensors, and this is the reason it exists. The two tools of the Wolfram Language for the job stop short of it. `ResourceFunction["EinsteinSummation"]` refuses an index that appears three times (`{{i}, {i}, {i}} -> {}`) and a batch index kept in the result (`{{k, a, b}, {k, b, c}} -> {k, a, c}`). `TensorContract` can sum over three or more slots, but of the outer product of all the operands, which it builds first, and it cannot keep a shared index in the result (a batch index, a diagonal).

2.1 One index in three tensors

```
Einsum["i,i,i->", Array[x, 2], Array[y, 2], Array[z, 2]]
```

$$x_1 y_1 z_1 + x_2 y_2 z_2$$

`TensorContract` gives the same sum from the outer product:

```
TensorContract[TensorProduct[Array[x, 2], Array[y, 2], Array[z, 2]], {{1, 2, 3}}] ==
 Einsum["i,i,i->", Array[x, 2], Array[y, 2], Array[z, 2]]
```

```
True
```

In the diagrams of section 4 such an index is a hyperedge, a dot joined to all the tensors that share it.

2.2 Batch indices

An index shared by the operands and kept in the result labels independent contractions, all done at once. The quadratic form $\sum_{ij} x_{bi} q_{ij} x_{bj}$ of every row b, with b in two operands and in the result:

```
rows = {{1, 0}, {0, 1}, {1, 1}}; q = {{2, 1}, {1, 3}};
Einsum["bi,ij,bj->b", rows, q, rows]
```

$$\{2, 3, 7\}$$

A thousand matrix products in one call:

```
SeedRandom[2]; xs = RandomReal[1, {1000, 3, 4}]; ys = RandomReal[1, {1000, 4, 2}];
Einsum["bij,bjk->bik", xs, ys] == MapThread[Dot, {xs, ys}]
```

```
True
```

2.3 The CP reconstruction

A tensor in the canonical polyadic form is a sum of outer products, $T_{abc} = \sum_i A_{ai} B_{bi} C_{ci}$: the index i is in three tensors, and a, b, c are in the result. With $r = 8$ terms:

```
SeedRandom[3]; {fa, fb, fc} = RandomReal[1, {3, 40, 8}];
cp = Einsum["ai,bi,ci->abc", fa, fb, fc];
reference = Sum[Outer[Times, fa[[All, i]], fb[[All, i]], fc[[All, i]]], {i, 8}];
{Dimensions[cp], Max[Abs[cp - reference]] < 10^-12}
```

```
{{40, 40, 40}, True}
```

`TensorContract` would first build the outer product of the three matrices, $40^3 \cdot 8^3$ numbers. Summa contracts two of them, keeping i as a batch index, and then the third, with half a million multiplications (the cost is given by `EinsumPath`, section 3):

```
{Times @@ Flatten[Dimensions /@ {fa, fb, fc}],
 EinsumPath["ai,bi,ci->abc", fa, fb, fc]["Flops"]}
```

$$\{32768000, 524800.\}$$

On the author's computer, with Mathematica 15, the outer product takes 262 MB and 0.37 s, the einsum 1.6 ms.

2.4 Shared and kept

An index can also be shared and kept without any other: the dot products of the rows of two matrices, $\sum_j m_{ij} n_{ij}$ for every i:

```
Einsum["ij,ij->i", m, n]
```

$$\{2, 7\}$$

3 The order of the contractions

3.1 What a path costs

An einsum of n tensors is computed as $n - 1$ contractions of two tensors, and their order decides the cost. `EinsumPath` tells which order the package chooses and what it costs, from the shapes alone (the tensors or their dimensions). For the product of three matrices, 1000×10, 10×1000 and 1000×10:

```
EinsumPath["ij,jk,kl->il", {{1000, 10}, {10, 1000}, {1000, 10}}]
```

```
<|"Spec" -> "ij,jk,kl->il", "Path" -> {{2, 3}, {1, 2}}, "Method" -> "Optimal",
 "Contractions" -> {"jk,kl->jl", "ij,jl->il"},
 "StepFlops" -> {100000., 100000.}, "Flops" -> 200000., "NaiveFlops" -> 2.*^8,
 "Speedup" -> 1000., "LargestIntermediate" -> 10000.|>
```

"Path" is in the linear format of numpy and opt_einsum [1], 1-based: each pair gives the positions of two tensors in the current list; they are removed, and their result is appended at the end. Here B and C go first, then A with their product. The cost of a step is the number of terms of its sum, the product of the dimensions of all its indices; "Flops" is the total, "NaiveFlops" the cost of the sum written out over all the indices at once, "Speedup" their ratio and "LargestIntermediate" the size of the largest tensor built on the way. Taking A times B first costs a hundred times more, and builds a 1000×1000 intermediate:

```
EinsumPath["ij,jk,kl->il", {{1000, 10}, {10, 1000}, {1000, 10}},
  "Path" -> {{1, 2}, {1, 2}}][[{"Contractions", "Flops"}]]
```

```
<|"Contractions" -> {"ij,jk->ik", "kl,ik->il"}, "Flops" -> 2.*^7|>
```

3.2 Strategies

"Path" -> "Optimal" finds the cheapest order exactly, by dynamic programming over the subsets of the tensors. The result of a subset keeps the indices still needed outside it or in the output, which is how an index shared by three tensors becomes a batch index of the intermediates. It is the default up to 8 tensors and is accepted up to 14; beyond 8 the default is "Greedy", the search of opt_einsum, which contracts at each step the pair that least increases the size. "Sequential" contracts from left to right, and a list of pairs is a path of your own. On a network of five tensors:

```
spec5 = "ab,ge,gac,dbf,abe->";
dims5 = {{50, 20}, {20, 50}, {20, 50, 5}, {10, 20, 20}, {50, 20, 50}};
AssociationMap[EinsumPath[spec5, dims5, "Path" -> #]["Flops"] &,
 {"Optimal", "Greedy", "Sequential", {{1, 5}, {1, 2}, {1, 2}, {1, 2}}}]
```

```
<|"Optimal" -> 110020., "Greedy" -> 1.03002*^6, "Sequential" -> 2.109*^6,
 {{1, 5}, {1, 2}, {1, 2}, {1, 2}} -> 161500.|>
```

3.3 The norm of a matrix product state

A matrix product state (MPS) of L sites has a tensor $A^{(k)}_{\alpha_{k-1}\sigma_k\alpha_k}$ on each site, with a physical index σ_k of dimension d and bonds α_k of dimension χ. Its norm $\langle\psi|\psi\rangle$ joins it to its conjugate over the physical indices. For four sites with $d = 2$ and $\chi = 64$ the sum over all the indices at once has $2^4 \cdot 64^6 \approx 7 \times 10^{10}$ terms of eight factors; the order chosen by the package costs 7×10^4 multiplications, a speed-up of 10^8:

```
mpsSpec = "pa,aqb,brc,cs,pA,AqB,BrC,Cs->";
mpsDims = {{2, 64}, {64, 2, 64}, {64, 2, 64}, {64, 2},
   {2, 64}, {64, 2, 64}, {64, 2, 64}, {64, 2}};
EinsumPath[mpsSpec, mpsDims][[{"Contractions", "Flops", "NaiveFlops", "Speedup",
   "LargestIntermediate"}]]
```

```
<|"Contractions" -> {"pa,aqb->pqb", "brc,cs->brs", "pqb,brs->pqrs",
    "pA,AqB->pqB", "BrC,Cs->Brs", "pqB,Brs->pqrs", "pqrs,pqrs->"},
 "Flops" -> 67600., "NaiveFlops" -> 7.69658*^12, "Speedup" -> 1.13855*^8,
 "LargestIntermediate" -> 256.|>
```

The order matters also among the reasonable ones. The function mps writes the norm of an MPS of

any length in the list syntax, with the indices `α[k]` (bonds of the ket), `β[k]` (bonds of the bra) and `σ[k]`; the tensors at the ends have one bond. `mpsShapes` gives their dimensions:

```
mps[L_] := Module[{ket = Table[{α[k - 1], σ[k], α[k]}, {k, L}]},
  ket[[1]] = Rest[ket[[1]]]; ket[[L]] = Most[ket[[L]]];
  Join[ket, ket /. α -> β] -> {}];
mpsShapes[L_, d_, χ_] := Join[#, #] &[Table[Which[k == 1, {d, χ}, k == L, {χ, d},
     True, {χ, d, χ}], {k, L}]];
mps[3]
```

```
{{σ[1], α[1]}, {α[1], σ[2], α[2]}, {α[2], σ[3]}, {σ[1], β[1]},
  {β[1], σ[2], β[2]}, {β[2], σ[3]}} -> {}
```

Contracting the ket first, in the order in which the tensors are written (`"Sequential"`), builds the whole state, d^L amplitudes: the cost grows exponentially with L. The order of the package keeps every intermediate small, as zipping ket and bra together site by site does, and its cost grows linearly; at twenty sites it is more than two thousand times cheaper:

```
TableForm[Table[{L, EinsumPath[mps[L], mpsShapes[L, 2, 16]]["Flops"],
    EinsumPath[mps[L], mpsShapes[L, 2, 16], "Path" -> "Sequential"]["Flops"]},
   {L, {4, 8, 12, 16, 20}}],
 TableHeadings -> {None, {"L", "Automatic", "Sequential"}}]
```

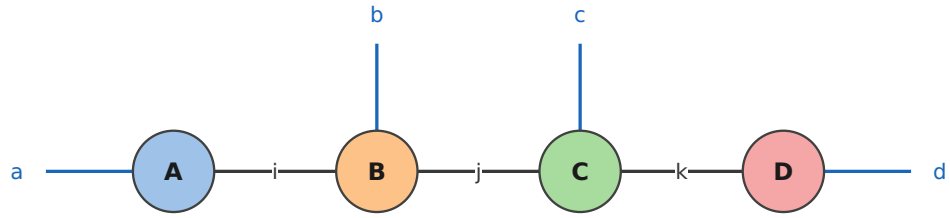
```
L    Automatic   Sequential
4    4624.       6688.
8    37120.      137248.
12   102656.     2.22621*^6
16   168192.     3.56496*^7
20   233728.     5.70423*^8
```

4 Pictures of a network

4.1 Bonds, open legs, traces, sums and hyperedges

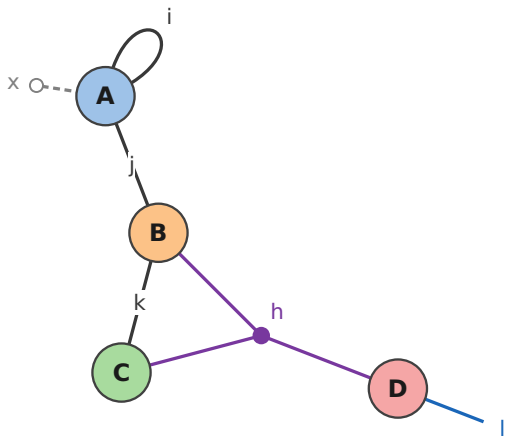
`EinsumDiagram` draws the network of an einsum: tensors are disks, a bond joins two tensors that share an index, and an index of the result is an open leg, in blue. The MPS of four sites, with its physical legs open:

```
EinsumDiagram["ai,ibj,jck,kd->abcd"]
```



Every index is drawn as a COPY tensor (a delta) with one leg for each time it appears, plus one if it is in the result. Two legs are a wire: a bond, an open leg, or a loop for a trace. One leg is an index summed in one tensor alone, drawn as a dashed leg ending in a small circle (a contraction with the vector of ones). Three or more legs are a hyperedge, a purple dot joined to all the tensors that share it. All of them in one network: the trace i and the summed index x on A, the bonds j and k, the hyperedge h shared by B, C and D, and the open leg l:

```
EinsumDiagram["iijx,jkh,kh,lh->l", ImageSize -> 300]
```

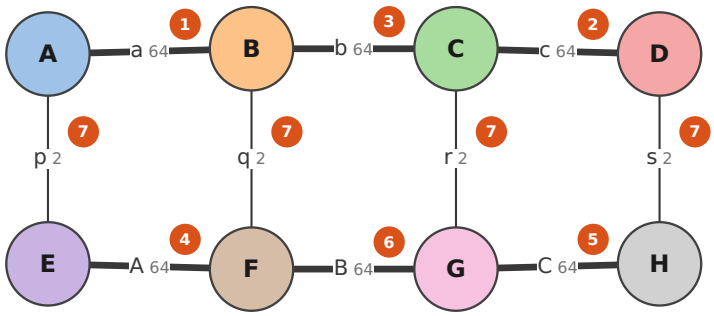


Tensors with the same name have the same colour, and tensors passed as symbols are named after them. The layout is computed by classical multidimensional scaling of the distances in the graph, refined by stress majorization [2]: chains come out straight, rings round and grids square.

4.2 Dimensions and the path

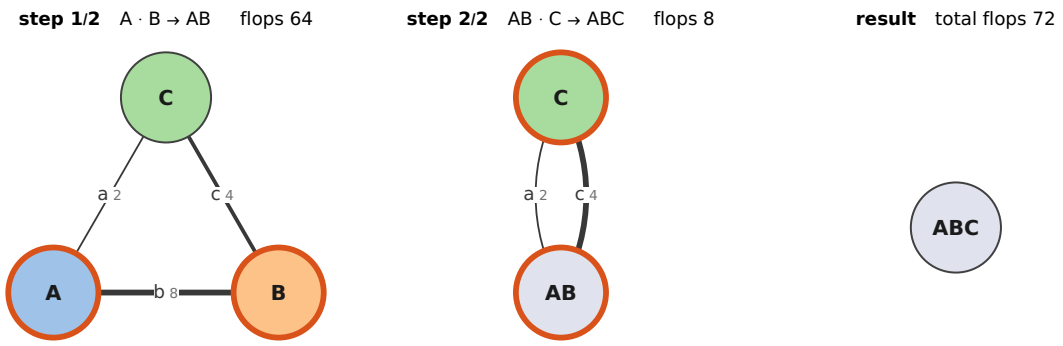
Given the dimensions, the diagram writes them next to the indices and draws thicker bonds for larger dimensions (on a logarithmic scale); `"ShowPath" -> True` marks each bond with the step at which it is contracted. The norm of the MPS of section 3.3: the ket is contracted from its two ends into its 16 amplitudes (steps 1 to 3), so is the bra (4 to 6), and the last step joins them over the physical indices; for four sites this is the cheapest order:

```
EinsumDiagram[mpsSpec, mpsDims, "ShowPath" -> True]
```



`"Steps" -> True` gives one picture per step, with the pair contracted next highlighted, all with the same plot range, so that `ListAnimate` shows the contraction. A ring of three tensors:

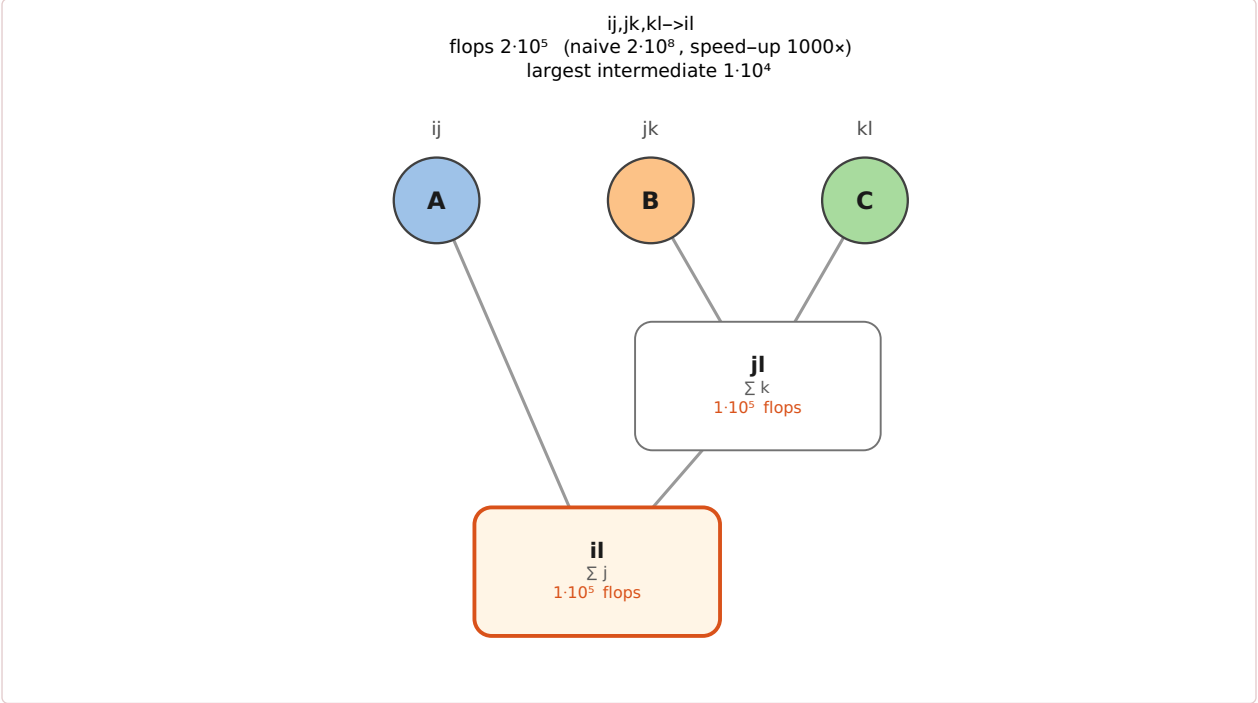
```
EinsumDiagram["ab,bc,ca->", {{2, 8}, {8, 4}, {4, 2}}, "Steps" -> True,
 ImageSize -> 220]
```



4.3 The contraction tree

`EinsumTree` draws the tree of the path: the leaves are the tensors, and each box is an intermediate, with its indices, the indices summed at that step and its cost; the title gives the total cost, the naive one, the speed-up and the largest intermediate. The matrix chain of section 3:

```
EinsumTree["ij,jk,kl->il", {{1000, 10}, {10, 1000}, {1000, 10}}, ImageSize -> 320]
```



5 Compiled einsums and the plan cache

`Einsum` keeps the plan of every call (the parsed specification, the path and the kernel of each step) in a cache, keyed by the specification and the shapes: a second call with the same shapes goes straight to the kernels. `EinsumFunction` makes the plan once, for given shapes, and gives a function to be applied in a loop. The transfer operator of an MPS, $x \mapsto \sum_s A^{s\,\mathsf{T}} x A^s$, that is $\sum_{ab} x_{ab} A_{asc} A_{bsd}$, has the summary of its plan:

```
SeedRandom[4]; tensorA = RandomReal[{-1, 1}, {8, 2, 8}];
transfer = EinsumFunction["ab,asc,bsd->cd", {{8, 8}, {8, 2, 8}, {8, 2, 8}}]
```

EinsumFunction[Spec: ab,asc,bsd–>cd Flops: 2048]

Its properties are those of the path:

```
{transfer["Contractions"], transfer["Flops"]}
```

```
{{"ab,bsd->asd", "asd,asc->cd"}, 2048.}
```

Three hundred applications of the transfer operator, normalized, converge to its dominant eigenvector: the eigenvalue is that of the 64×64 transfer matrix, built by another einsum:

```
step[x_] := #/Norm[#] &[transfer[x, tensorA, tensorA]];
fixedPoint = Nest[step, IdentityMatrix[8], 300];
transferMatrix = ArrayReshape[Einsum["asc,bsd->abcd", tensorA, tensorA], {64, 64}];
{Norm[transfer[fixedPoint, tensorA, tensorA]],
 Max[Abs[Eigenvalues[transferMatrix]]]}
```

$$\{4.79617, 4.79617\}$$

`EinsumClearCache[]` empties the cache.

6 Symbolic, exact, complex and sparse arrays

The kernels are `Dot`, `Transpose`, `Total` and products of arrays, so the entries can be anything: symbols, exact numbers, complex numbers. A product of symbolic matrices:

```
MatrixForm[Einsum["ij,jk->ik", Array[a, {2, 2}], Array[b, {2, 2}]]]
```

$$\begin{pmatrix} a_{11}b_{11} + a_{12}b_{21} & a_{11}b_{12} + a_{12}b_{22} \\ a_{21}b_{11} + a_{22}b_{21} & a_{21}b_{12} + a_{22}b_{22} \end{pmatrix}$$

The cross product, with the Levi-Civita tensor `LeviCivitaTensor[3]` (a `SparseArray`), and the identity $\sum_i \epsilon_{ijk}\epsilon_{imn} = \delta_{jm}\delta_{kn} - \delta_{jn}\delta_{km}$:

```
Einsum["ijk,j,k->i", LeviCivitaTensor[3], Array[u, 3], Array[v, 3]]
```

$$\{u_2 v_3 - u_3 v_2, u_3 v_1 - u_1 v_3, u_1 v_2 - u_2 v_1\}$$

```
With[{ϵ = LeviCivitaTensor[3], δ = IdentityMatrix[3]},
 Einsum["ijk,imn->jkmn", ϵ, ϵ] ==
  Einsum["jm,kn->jkmn", δ, δ] - Einsum["jn,km->jkmn", δ, δ]]
```

```
True
```

The Pauli matrices as one $3 \times 2 \times 2$ array: $\sum_a \sigma^a \sigma^a = 3$, and $\sigma^a \sigma^b = \delta_{ab} + i \sum_c \epsilon_{abc} \sigma^c$:

```
pauli = PauliMatrix /@ {1, 2, 3};
MatrixForm[Einsum["aij,ajk->ik", pauli, pauli]]
```

$$\begin{pmatrix} 3 & 0 \\ 0 & 3 \end{pmatrix}$$

```
Einsum["aij,bjk->abik", pauli, pauli] ==
 Einsum["ab,ik->abik", IdentityMatrix[3], IdentityMatrix[2]] +
  I Einsum["abc,cik->abik", LeviCivitaTensor[3], pauli]
```

```
True
```

A `SparseArray` stays sparse in the steps that are matrix products; the other kernels make it dense. The square of the hopping matrix of a ring of 1000 sites:

```
ring = SparseArray[{Band[{1, 2}] -> 1., Band[{2, 1}] -> 1., {1, 1000} -> 1.,
    {1000, 1} -> 1.}, {1000, 1000}];
ring2 = Einsum["ij,jk->ik", ring, ring];
{Head[ring2], Length[ring2["NonzeroValues"]]}
```

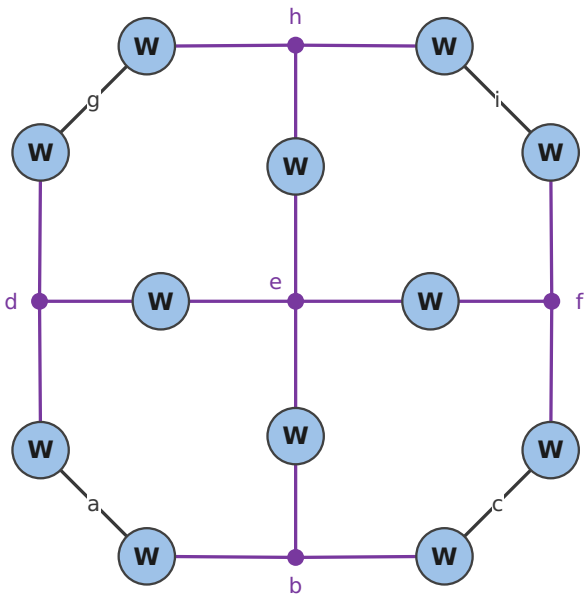
```
{SparseArray, 3000}
```

7 Physics

7.1 The partition function of the Ising model

On a lattice, $Z = \sum_{\{s\}} \prod_{\langle ij \rangle} W_{s_i s_j}$ with $W_{ss'} = e^{\beta J s s'}$: each bond is a 2×2 matrix, and each spin an index shared by all the bonds that touch it. On an open 3×3 lattice with the sites $a, \ldots, i$ the inner spins are in three or four bonds, the hyperedges of the diagram, and the spins at the corners, in two bonds, are plain bonds:

```
lattice = "ab,bc,de,ef,gh,hi,ad,be,cf,dg,eh,fi->";
EinsumDiagram[lattice, "TensorNames" -> ConstantArray["W", 12], ImageSize -> 340]
```



With $x = e^{\beta J}$, the partition function is a polynomial in x and $1/x$, exact:

```
boltzmann = {{x, 1/x}, {1/x, x}};
zIsing = Expand[Einsum[lattice, Sequence @@ ConstantArray[boltzmann, 12]]]
```

$$2x^{12} + \frac{2}{x^{12}} + 8x^8 + \frac{8}{x^8} + 32x^6 + \frac{32}{x^6} + 46x^4 + \frac{46}{x^4} + 96x^2 + \frac{96}{x^2} + 144$$

It is the sum over the 2^9 configurations:

```
bonds = Characters /@ StringSplit[StringDrop[lattice, -2], ","];
weight[s_] := With[{c = AssociationThread[Characters["abcdefghi"], s]},
   Times @@ (x^(c[#1] c[#2]) & @@@ bonds)];
Expand[zIsing - Sum[weight[s], {s, Tuples[{1, -1}, 9]}]]
```

$$0$$

7.2 Matrix product states, exactly

The AKLT state of the spin-1 chain [3] is an MPS with $\chi = 2$: $A^{+1} = \sqrt{2/3}\,\sigma^{+}$, $A^{0} = -\sqrt{1/3}\,\sigma^{z}$, $A^{-1} = -\sqrt{2/3}\,\sigma^{-}$. On a ring of L sites, `akltRing` writes its norm, a trace over the bonds, and with operators on some sites an expectation value. With exact entries the results are exact:

```
aklt = Transpose[{Sqrt[2/3] {{0, 1}, {0, 0}}, -Sqrt[1/3] {{1, 0}, {0, -1}},
    -Sqrt[2/3] {{0, 0}, {1, 0}}}, {2, 1, 3}];
akltRing[L_, ops_ : {}] := Join[
   Table[{α[k], σ[k], α[Mod[k, L] + 1]}, {k, L}],
   Table[{β[k], If[MemberQ[ops, k], τ[k], σ[k]], β[Mod[k, L] + 1]}, {k, L}],
   ({σ[#], τ[#]} &) /@ ops] -> {};
Table[Einsum[akltRing[L], Sequence @@ ConstantArray[aklt, 2 L]], {L, 2, 7}]
```

$$\left\{\frac{4}{3}, \frac{8}{9}, \frac{28}{27}, \frac{80}{81}, \frac{244}{243}, \frac{728}{729}\right\}$$

They are $\operatorname{Tr} E^{L} = 1 + 3(-1/3)^{L}$, from the eigenvalues of the transfer matrix E, 1 and $-1/3$ three times:

```
Table[1 + 3 (-1/3)^L, {L, 2, 7}]
```

$$\left\{\frac{4}{3}, \frac{8}{9}, \frac{28}{27}, \frac{80}{81}, \frac{244}{243}, \frac{728}{729}\right\}$$

The correlation $\langle S^{z}_{1} S^{z}_{1+r}\rangle$ on a ring of ten sites, for $r = 1, \dots, 4$; as the ring grows it tends to its value on the infinite chain, $\frac{4}{3}(-\frac{1}{3})^{r}$, that is $-\frac{4}{9}$, $\frac{4}{27}$, $-\frac{4}{81}$, $\frac{4}{243}$:

```
sz = DiagonalMatrix[{1, 0, -1}];
Table[Einsum[akltRing[10, {1, 1 + r}], Sequence @@ ConstantArray[aklt, 20], sz, sz]/
   Einsum[akltRing[10], Sequence @@ ConstantArray[aklt, 20]], {r, 4}]
```

$$\left\{-\frac{6562}{14763}, \frac{730}{4921}, -\frac{246}{4921}, \frac{90}{4921}\right\}$$

7.3 A Kubo formula

The optical conductivity of a two-band chain, the Rice–Mele model [4] $H(k) = (1 + 0.6\cos k)\,\sigma_x + 0.6\sin k\,\sigma_y + 0.4\,\sigma_z$, at half filling. With the band energies $E_n(k)$, the occupations $f_n(k)$ and the current $J(k) = \partial_k H(k)$ in the band basis, $J_{nm}(k) = \langle nk|J(k)|mk\rangle$, the Kubo formula reads

$$\sigma(\omega) \propto \frac{i}{N} \sum_{k} \sum_{nm} J_{nm}(k)\, J_{mn}(k)\, \frac{f_n(k) - f_m(k)}{E_m(k) - E_n(k)}\, \frac{1}{\omega - [E_m(k) - E_n(k)] + i\eta}.$$

The bands on $N = 400$ values of k:

```
nk = 400; ks = 2. Pi Range[0, nk - 1]/nk; ωs = Range[0., 4., 0.01]; η = 0.05;
h[k_] := (1 + 0.6 Cos[k]) PauliMatrix[1] + 0.6 Sin[k] PauliMatrix[2] +
   0.4 PauliMatrix[3];
current[k_] := -0.6 Sin[k] PauliMatrix[1] + 0.6 Cos[k] PauliMatrix[2];
bands[k_] := Transpose[SortBy[Transpose[Eigensystem[h[k]]], First]];
{energies, states} = Transpose[bands /@ ks];
Dimensions /@ {energies, states}
```

```
{{400, 2}, {400, 2, 2}}
```

The currents in the band basis, $J_{nm}(k) = \sum_{ij} \bar{u}_{ni}(k) J_{ij}(k) u_{mj}(k)$, for all k in one einsum, where k is in three tensors:

```
bandCurrents = Einsum["kni,kij,kmj->knm", Conjugate[states], current /@ ks, states];
Dimensions[bandCurrents]
```

$$\{400, 2, 2\}$$

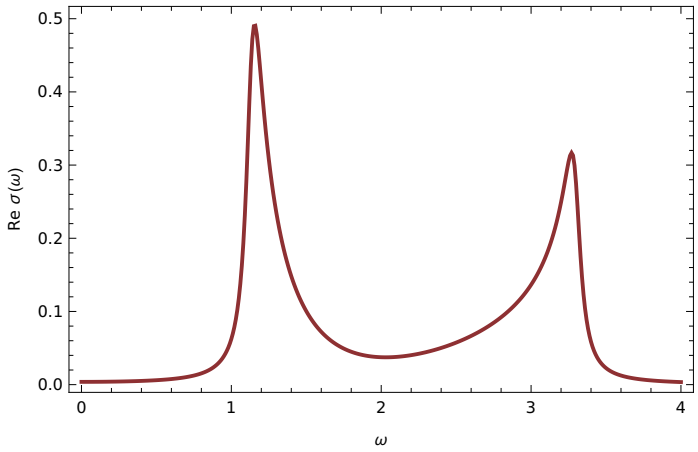
The last two factors of the formula are an array $W_{knm\omega}$, on 401 frequencies (zero for $n = m$):

```
f = UnitStep[-energies]; e = energies;
weights = Table[With[{de = e[[k, m]] - e[[k, n]]},
    If[n == m, 0. ωs, (f[[k, n]] - f[[k, m]])/de/(ωs - de + I η)]],
   {k, nk}, {n, 2}, {m, 2}];
Dimensions[weights]
```

$$\{400, 2, 2, 401\}$$

The formula is then one einsum, written as it reads: k, n and m each appear in three tensors. The absorption, $\mathrm{Re}\,\sigma(\omega)$, starts at the gap, $2\min_k E_2(k) \approx 1.13$, and ends at $2\max_k E_2(k) \approx 3.30$, with the peaks of the van Hove singularities at the edges of the band of transitions:

```
conductivity = I/nk Einsum["knm,kmn,knmw->w", bandCurrents, bandCurrents, weights];
ListLinePlot[Transpose[{ωs, Re[conductivity]}], Frame -> True,
 FrameLabel -> {"ω", "Re σ(ω)"}, PlotStyle -> RGBColor["#8E3032"],
 PlotRange -> All, ImageSize -> 360]
```



It is the sum written out:

```
writtenOut = I/nk Sum[
    bandCurrents[[k, n, m]] bandCurrents[[k, m, n]] weights[[k, n, m]],
    {k, nk}, {n, 2}, {m, 2}];
Max[Abs[conductivity - writtenOut]] < 10^-12
```

```
True
```

The package computes it in two steps, the product of the currents and its contraction with the weights:

```
EinsumPath["knm,kmn,knmw->w", bandCurrents, bandCurrents, weights][[
 {"Contractions", "Flops", "NaiveFlops"}]]
```

```
<|"Contractions" -> {"knm,kmn->knm", "knm,knmw->w"}, "Flops" -> 643200.,
 "NaiveFlops" -> 1.2832*^6|>
```

7.4 A loop of propagators

With the Green's functions $G_k(z) = [z - H(k)]^{-1}$, the trace of a loop of three, $\mathrm{Tr}\, G_k(z_1)G_k(z_2)G_k(z_3)$ for every k, as in a triangle diagram of nonlinear response, is `"kab,kbc,kca->k"`. In the band basis it is $\sum_n \prod_i [z_i - E_n(k)]^{-1}$:

```
propagators[z_] := Inverse[z IdentityMatrix[2] - h[#]] & /@ ks;
{z1, z2, z3} = {0.3 + 0.1 I, -0.2 + 0.2 I, 1. + 0.1 I};
loop = Einsum["kab,kbc,kca->k", propagators[z1], propagators[z2], propagators[z3]];
Max[Abs[loop - Total[1/((z1 - e) (z2 - e) (z3 - e)), {2}]]] < 10^-12
```

```
True
```

8 How a contraction runs

Each pairwise contraction is classified by its indices: batch indices (in both operands and kept), contracted indices (in both and summed) and free indices (in one operand). Before that, the indices that appear in one operand only and are not needed afterwards are summed with `Total`, and diagonals are taken by one `Transpose` with repeated levels. Then one kernel runs:

- **Dot**: no batch index. Each operand becomes a matrix with one `Transpose` and one `ArrayReshape` at most, none when its indices are already in order, and the product is one call of BLAS. Among the orders of the operands and of their indices, the one that moves the least data, the final result included, is chosen.
- **Batch**: batch indices, $C_{bik} = \sum_j L_{bij} R_{bjk}$. A cost model chooses between a loop of `Dot` (large matrices) and a vectorised product with `Total` on packed arrays (many small matrices).
- **Reduce**: two operands with the same indices, as `"kh,kh->h"`: the product in the layout of one of them and `Total` over the contracted levels, instead of transposing both.
- **Hadamard**, **Prefix**, **Suffix**: no summed index; the elementwise product, or a broadcasting product when the indices of one operand are the first or the last of the other. **Scale**: a scalar operand.

The plan of an `EinsumFunction` tells the kernel of each step (and `Dot` or `Vec`, the vectorised product, for a batch):

```
kernel[s_] := s["Type"] <>
   If[KeyExistsQ[s, "Method"], " (" <> s["Method"] <> ")", ""];
kernels[spec_, dims_] := {spec, ToString[dims],
   StringRiffle[kernel /@ EinsumFunction[spec, dims]["Plan"]["Steps"], ", "]};
TableForm[kernels @@@ {{"ij,jk->ik", {{300, 300}, {300, 300}}},
    {"ijk,kj->i", {{40, 40, 40}, {40, 40}}},
    {"bij,bjk->bik", {{200, 64, 64}, {200, 64, 64}}},
    {"bij,bjk->bik", {{20000, 4, 4}, {20000, 4, 4}}},
    {"kh,kh->h", {{100, 100}, {100, 100}}}, {"ij,j->ij", {{100, 100}, {100}}},
    {"ijk,ijk,ijk->", {{10, 10, 10}, {10, 10, 10}, {10, 10, 10}}}},
 TableHeadings -> {None, {"spec", "dimensions", "kernels"}}]
```

spec	dimensions	kernels
ij,jk->ik	{{300, 300}, {300, 300}}	Dot
ijk,kj->i	{{40, 40, 40}, {40, 40}}	Dot
bij,bjk->bik	{{200, 64, 64}, {200, 64, 64}}	Batch (Dot)
bij,bjk->bik	{{20000, 4, 4}, {20000, 4, 4}}	Batch (Vec)
kh,kh->h	{{100, 100}, {100, 100}}	Reduce
ij,j->ij	{{100, 100}, {100}}	Suffix
ijk,ijk,ijk->	{{10, 10, 10}, {10, 10, 10}, {10, 10, 10}}	Hadamard, Dot

The flops count the arithmetic; the kernels decide how fast it goes. A `Dot` runs at the speed of BLAS, and a batch of $2 \cdot 10^4$ products of 4×4 matrices runs as a few operations on packed arrays, not as $2 \cdot 10^4$ calls. Intermediates are freed as soon as they are used. `Examples/Benchmarks.wl` measures the timings on your computer, against `Dot`, `MapThread`, `TensorContract` and `Total`.

9 How it is checked

The test suite (`wolfram -script Tests/RunTests.wls`, 735 tests) checks 213 einsums against `numpy.einsum`, computed in exact integer arithmetic: 63 chosen to cover every kernel, repeated with every path strategy and both batch methods, and 150 random ones. On 40 networks it checks that the cost of the `"Optimal"` path equals that of an exhaustive search over all the orders, and that `"Greedy"` is never cheaper. It compares results in machine precision, real and complex, with sums written out, and checks symbolic and sparse inputs, the syntaxes, the messages, `EinsumFunction`, the diagrams and the update check. The package was first written without Mathematica and checked with Mathics3, an open-source interpreter of the Wolfram Language, against `numpy.einsum` on about 2500 einsums, with every path and every kernel.

The exhaustive search is easy to repeat: the five tensors of section 3 can be contracted in $10 \cdot 6 \cdot 3 = 180$ orders, and the cheapest one costs as much as the one the package finds:

```
allPaths[1] = {{}};
allPaths[k_] := Join @@ Table[Prepend[#, p] & /@ allPaths[k - 1],
   {p, Subsets[Range[k], {2}]}];
costs = EinsumPath[spec5, dims5, "Path" -> #]["Flops"] & /@ allPaths[5];
{Length[costs], Min[costs], EinsumPath[spec5, dims5]["Flops"]}
```

```
{180, 110020., 110020.}
```

And a ring of three tensors, in integers, against its sum written out:

```
SeedRandom[5]; {ta, tb, tc} = RandomInteger[{-5, 5}, {3, 3, 4, 3}];
Einsum["iaj,jbk,kci->abc", ta, tb, tc] ==
 Table[Sum[ta[[i, a, j]] tb[[j, b, k]] tc[[k, c, i]], {i, 3}, {j, 3}, {k, 3}],
  {a, 4}, {b, 4}, {c, 4}]
```

```
True
```

References

[1] D. G. A. Smith and J. Gray, J. Open Source Softw. **3**, 753 (2018).

[2] E. R. Gansner, Y. Koren, and S. North, in *Graph Drawing 2004*, Lect. Notes Comput. Sci. **3383**, 239 (Springer, 2005).

[3] I. Affleck, T. Kennedy, E. H. Lieb, and H. Tasaki, Phys. Rev. Lett. **59**, 799 (1987).

[4] M. J. Rice and E. J. Mele, Phys. Rev. Lett. **49**, 1455 (1982).